WEB SYSTEMS DESIGN – PROJECT 9

VISTULA

UNIVERSITY

Name: Amjad Massaoud

Student ID: 78706

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

1. Introduction

This report presents the implementation of Project 9 for the Web Systems Design course. The project extends the existing Laravel-based web system with a simplified News Feed Service, which is one of the central components of modern social-media platforms. The service allows one user to follow another user, unfollow a user, and request a personalized feed containing photos from followed accounts.

The implementation continues the architecture developed in the previous laboratories. The backend is built with Laravel and exposes structured JSON APIs under the versioned student API prefix `/api/78706/v1`. PostgreSQL stores application data, Redis is used as a caching layer, and Docker Compose runs the backend, database, Redis, and web proxy services together. Project 9 adds the social graph and feed aggregation layer on top of the existing Photo Upload Service created in Project 8.

2. Project Assumptions and Objectives

The main assumption of the project is that the feed is generated from follow relationships. In this laboratory version, the user identity is simplified to a static user ID equal to 1. In a production system, the same logic would normally be connected to authenticated sessions or API tokens.

The main objectives were the following:

- Create a follows table with follower_id and followed_id fields.
- Prevent duplicate follow relationships by using a unique pair constraint.
- Create a Follow model for Eloquent persistence.
- Extend the User model with following() and followers() relationships.
- Create a FollowController with follow and unfollow operations.
- Invalidate cached feed data after follow and unfollow actions.
- Create a FeedController that retrieves followed users, aggregates their photos, removes duplicates, and returns JSON data.
- Use Redis-based Laravel caching for feed fragments.
- Use cursor pagination to support infinite-scroll-ready feed responses.
- Register versioned API routes under `/api/78706/v1`.
- Verify the implementation using route:list, curl commands, Docker Compose, and Redis commands.

3. Technological Foundations

Technology	Role in Project 9
Laravel 12	Backend API framework, routing, controllers, Eloquent ORM, cache facade, and pagination.
PostgreSQL	Relational database used to store users, photos, and follow relationships.
Redis	Caching layer used for storing feed fragments and improving repeated feed response performance.
Docker Compose	Runs the backend, web proxy, PostgreSQL database, and Redis containers together.
Nginx / Web service	Exposes the application externally on port 8080 and forwards API traffic to the backend.
Cursor Pagination	Supports infinite-scroll-style pagination without expensive offset queries.
Git Bash & VS Code	Used for command execution, testing, code editing, and screenshot evidence.

4. Implementation Process

The implementation was completed in several phases. First, the baseline Docker services were verified. Then the follows migration and Follow model were created. After that, the User model was extended with social graph relationships. The follow and unfollow operations were implemented in FollowController. Finally, the feed generation logic was implemented in FeedController, and the required API routes were added under the versioned student prefix.

The feed endpoint uses a fan-out-on-read approach. When the client requests the feed, the backend retrieves the accounts followed by user 1, queries photos belonging to those accounts, orders the results by creation date and ID, applies cursor pagination, and returns a JSON response. Cache::remember() is used to cache feed results for 60 seconds. When a follow or unfollow operation occurs, Cache::flush() invalidates stale feed data.

Figure 1. Docker Compose verification showing the backend, PostgreSQL database, Redis, and web containers running.

```
jad@fedora-2:~/uni-project/project9$ docker compose ps
NAME                IMAGE                  SERVICE         STATUS                  PORTS
wsd_backend         wsd-project-backend    backend         Up 5 minutes            0.0.0.0:8000-
>8000/tcp
wsd_db              postgres:16-alpine     db              Up 58 minutes           5432/tcp
wsd_redis           redis:7-alpine         redis           Up 58 minutes           6379/tcp
wsd_web             wsd-project-web        web             Up 58 minutes           0.0.0.0:8080-
>80/tcp
```

Figure 1. Docker Compose verification showing all services active.

Figure 3. Migration status output showing existing database migrations and the create_follows_table migration entry.

```
jad@fedora-2:~/uni-project/project9/backend$ php artisan migrate:status
2026_08_30_250000_create_follows_table ..................................... [5] Ran
```

Figure 3. Migration status showing create_follows_table marked as Ran.

Figure 4. Follow model with fillable follower_id and followed_id fields.

Follow.php — backend > app > Models > Follow.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Follow extends Model
{
    protected $fillable = [
        'follower_id',
        'followed_id',
    ];
}
```

Figure 4. Follow model with fillable follower_id and followed_id fields.

Figure 5. User model relationships: following() and followers() methods defining the social graph.

User.php — backend > app > Models > User.php

```php
    public function following(): BelongsToMany
    {
        return $this->belongsToMany(
            User::class,
            'follows',
            'follower_id',
            'followed_id'
        );
    }

    public function followers(): BelongsToMany
    {
        return $this->belongsToMany(
            User::class,
            'follows',
            'followed_id',
            'follower_id'
        );
    }
```

Figure 5. User model relationships defining the social graph.

Figure 6. FollowController implementation: follow method, duplicate prevention, self-follow protection, record creation, and cache invalidation.

FollowController.php — app > Http > Controllers > Api > FollowController.php

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Follow;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Cache;

class FollowController extends Controller
{
    public function follow(int $id): JsonResponse
    {
        $followerId = 1;

        if ($followerId === $id) {
            return response()->json([
                'message' => 'You cannot follow yourself'
            ], 422);
        }

        $exists = Follow::query()
            ->where('follower_id', $followerId)
            ->where('followed_id', $id)
            ->exists();

        if ($exists) {
            return response()->json([
                'message' => 'Already following'
            ], 409);
        }

        Follow::create([
            'follower_id' => $followerId,
            'followed_id' => $id,
        ]);

        Cache::flush();

        return response()->json([
            'message' => 'Followed successfully'
        ], 201);
    }
```

Figure 6. FollowController follow method with validation and cache invalidation.

Figure 7. FollowController implementation: unfollow method and cache invalidation after deleting the follow relationship.

FollowController.php

```
45      public function unfollow(int $id): JsonResponse
46      {
47          $followerId = 1;
48
49          Follow::query()
50              ->where('follower_id', $followerId)
51              ->where('followed_id', $id)
52              ->delete();
53
54          Cache::flush();
55
56          return response()->json([
57              'message' => 'Unfollowed successfully'
58          ], 200);
59      }
60  }
```

Figure 7. FollowController unfollow method and cache clearing logic.

Figure 8. FeedController implementation: request limit handling, cache key creation, retrieval of followed accounts, and early empty-feed response.

FeedController.php — app > Http > Controllers > Api > FeedController.php

```php
    public function index(Request $request): JsonResponse
    {
        $userId = 1;

        $limit = (int) $request->query('limit', 10);
        $limit = max(1, min($limit, 50));

        $cursor = (string) $request->query('cursor', 'first');
        $cacheKey = 'feed:' . $userId . ':' . $limit . ':' . $cursor;

        $result = Cache::remember($cacheKey, 60, function () use ($userId, $limit) {
            $followedIds = Follow::query()
                ->where('follower_id', $userId)
                ->pluck('followed_id')
                ->unique()
                ->values();

            if ($followedIds->isEmpty()) {
                return [
                    'count' => 0,
                    'data' => [],
                    'next_cursor' => null,
                    'prev_cursor' => null,
                ];
            }
```

Figure 8. FeedController request limit handling, cache key creation, and early empty-feed response.

Figure 9. FeedController implementation: photo aggregation, chronological ordering, cursor pagination, cursor values, and JSON response.

FeedController.php

```php
            $paginator = Photo::query()
                ->whereIn('user_id', $followedIds)
                ->orderByDesc('created_at')
                ->orderByDesc('id')
                ->cursorPaginate($limit);

            return [
                'count' => count($paginator->items()),
                'data' => $paginator->items(),
                'next_cursor' => optional($paginator->nextCursor())->encode(),
                'prev_cursor' => optional($paginator->previousCursor())->encode(),
            ];
        });

        return response()->json($result, 200);
    }
```

Figure 9. FeedController cursor pagination and JSON response structure.

Figure 10. Final api.php route registration for Project 9 inside the versioned API group.

api.php — backend > routes > api.php

```
29  // Project 9 routes
30  Route::post('users/{id}/follow', [FollowController::class, 'follow']);
31  Route::delete('users/{id}/follow', [FollowController::class, 'unfollow']);
32  Route::get('feed', [FeedController::class, 'index']);
33
```

Figure 10. Final api.php route registration for Project 9.

Figure 11. Laravel route:list proof showing the registered feed, follow, unfollow, photos, restaurants, short-links, and tasks endpoints.

```
jad@fedora-2:~/uni-project/project9/backend$ php artisan route:list
GET|HEAD   api/78706/v1/feed ..............................  Api\FeedController@index
POST       api/78706/v1/users/{id}/follow .................  Api\FollowController@follow
DELETE     api/78706/v1/users/{id}/follow .................  Api\FollowController@unfollow
GET|HEAD   api/78706/v1/photos ............................  photos.index › Api\PhotoController@index
GET|HEAD   api/78706/v1/restaurants/nearby ................  Api\RestaurantController@nearby
GET|HEAD   api/78706/v1/tasks .............................  tasks.index › Api\TaskController@index
Showing [50] routes
```

Figure 11. Route list showing the feed and follow endpoints registered under /api/78706/v1.

Figure 12. Redis DBSIZE command output used to verify the Redis service state during testing.

```
jad@fedora-2:~/uni-project/project9$ docker compose exec redis redis-cli DBSIZE
(integer) 0
```

Figure 12. Redis DBSIZE verification during testing.

4.2 Conceptual Explanations

The follows table represents the social graph. The follower_id column identifies the user who follows another account, while followed_id identifies the account being followed. The unique constraint on the pair prevents the same user from following the same account more than once.

The User model relationships make the social graph easier to use in Laravel. The following() method returns the accounts a user follows, while followers() returns the accounts that follow that user. These relationships are important because a feed service depends on knowing which accounts should contribute content to the user feed.

The feed service is computationally expensive because it must identify followed users, aggregate posts from multiple accounts, rank or order those posts, remove duplicates, paginate the result, and return the response quickly. In large social platforms, this process is performed for millions of users and billions of posts, so caching and careful pagination are necessary.

This implementation uses fan-out-on-read. The feed is built when the user requests it. This is simpler and easier to keep consistent for a laboratory project. In a fan-out-on-write architecture, the system would push each new post into the feed inboxes of followers at write time. Fan-out-on-write can make feed reads very fast, but it is more complex and expensive when a user has many followers.

Redis caching reduces repeated feed generation work. The controller builds a cache key based on user ID, result limit, and cursor information. Cache::remember() stores the generated feed for 60 seconds. When a follow or unfollow operation changes the social graph, Cache::flush() is called so old feed results are not reused.

Cursor pagination is used because it is better suited for infinite scrolling than offset pagination. Offset pagination becomes slower as the offset grows and can also produce inconsistent results when new posts are inserted. Cursor pagination continues from a stable position in the ordered dataset, which makes it more scalable for feeds.

Concept	Explanation
Social graph	Follow relationships define which users contribute content to the feed.
Deduplication	Duplicate content must be removed so the same post is not shown multiple times.
Caching	Feed fragments are cached to reduce database work and improve response time.
Pagination	The API returns limited chunks of the feed rather than all posts at once.
Infinite scroll	Cursor pagination supports loading the next part of the feed efficiently.
Ranking	This project uses chronological ordering; production systems may use AI and engagement-based ranking.
Synchronization	Follow/unfollow operations invalidate cache so the feed stays consistent with the social graph.

4.3 Troubleshooting Technical Issues

1. Route Registration & Cache: During implementation, clearing route caches and rebuilding the backend container ensured that the new follow and feed routes were accurately recognized by Laravel.

2. Migration Consistency: The follows table was migrated and confirmed with migrate:status to ensure foreign key and unique pair constraints were active before running feature tests.

3. Redis State & Invalidation: Cache invalidation on follow/unfollow actions was verified to prevent stale feeds after social graph modifications.

5. Conclusion

Project 9 successfully extends the existing web system with a simplified News Feed Service. The implementation adds follow relationships, a Follow model, user social graph relationships, follow and unfollow controller actions, a feed aggregation controller, versioned API routes, cursor pagination, chronological ranking, and cache invalidation logic.

The screenshots document the major implementation evidence: Docker services running, route registration in api.php, follow model creation, user relationships, FollowController logic, FeedController logic, Laravel route:list verification, and Redis inspection. The project reflects core distributed-systems concepts discussed in the laboratory: personalized feeds, fan-out-on-read, caching, pagination, synchronization, deduplication, and simplified ranking.

6. References

1. Laravel Documentation. Pagination: https://laravel.com/docs/pagination
2. Laravel Documentation. Cache: https://laravel.com/docs/cache
3. Laravel Documentation. Eloquent Relationships: https://laravel.com/docs/eloquent-relationships
4. Laravel Documentation. Eloquent ORM: https://laravel.com/docs/eloquent
5. Redis Documentation: https://redis.io/docs/latest/
6. Docker Documentation. Compose overview: https://docs.docker.com/compose/
7. PostgreSQL Documentation: https://www.postgresql.org/docs/